

Merge as Graph Extension

Sasson Margaliot

June 2026

Abstract

This article proposes a formal interpretation of Merge as graph extension inside the Workspace. The Workspace is represented as a graph: elementary syntactic objects are nodes, and complex objects formed by Merge are accessible paths or subgraphs between boundary nodes. When Merge applies, it does not erase the previous Workspace; it adds a new accessible path that can participate in later applications of Merge.

The same processing cycle is proposed for five internal Layers of the Language Faculty: Semantic Component (Σ), A-Syntax, A-bar Syntax, Distributed Morphology (DM), and Phonological Component (Φ).

Each Layer applies an ordered grammar of Extension Rules, relinearizes the result, and transfers a new graph to the next Layer. The Layers are traversed in one order in generation and in the reverse order in comprehension, while the basic Expand algorithm remains the same.

Version 2: stylistic revisions, terminology adjustments, and typo corrections.

1. Introduction

The proposal developed here is that the Workspace is represented as a graph, and Merge can be formally interpreted as graph extension. When Merge applies, it adds a new accessible path to that graph. The material already present in the Workspace is preserved, and the newly added path can itself participate in later applications of Merge.

In standard linguistic notation, a rule such as

(1) $VP \Rightarrow V NP$

is naturally understood as a parsing rule: when a $V NP$ configuration is found, it may be recognized as a VP .

An Extension Rule written as $Left \Rightarrow Right$ is applied by finding the $Right$ side in the Workspace and adding the $Left$ side as a new accessible path.

Each Layer has its own ordered grammar of Extension Rules. The Layer receives a Workspace, applies Merge repeatedly inside that Workspace, and then produces a new graph through Relinearization. Transfer passes that graph as the initial Workspace of the next Layer.

The same Expand algorithm is proposed for five internal Layers of the Language Faculty, traversed in one order in generation and in the reverse order in comprehension. The grammars differ from Layer to Layer, but the basic processing cycle remains the same.

The article is organized as follows. Section 2 presents the proposed five-Layer architecture of the Language Faculty. The notion of Workspace is then developed in Sections 3–4, with the directed-graph notation introduced in Section 5. Section 6 defines Extension Rules and gives the first simple example of Merge as the addition of a new path. A larger Workspace is considered in Section 7. Section 8 turns to Layer grammars and the Expand procedure. The central interpretation of Merge as graph extension is stated explicitly in Section 9. Relinearization and Transfer are discussed in Section 10. Section 11 concludes.

2. The five internal Layers

The proposal assumes five Layers internal to the Language Faculty:

- Semantic Component (Σ),
- A-Syntax,
- A-bar Syntax,
- Distributed Morphology (DM),
- Phonological Component (Φ)

These five Layers are distinct from the external systems. C-I is the intelligent Agent itself, interacting with other cognitive systems. A-P is the biological articulatory-perceptual performance system. The five Layers are the internal linguistic machinery between them.

This article proposes this five-Layer organization as the internal architecture of the Language Faculty. The full *generation* pipeline is:

(2)

C-I / Agent → [Agentic Form] →

Semantic Component (Σ) → [Event Structure] →

A-Syntax → [NP-Structure] →

A-bar Syntax → [Surface Structure] →

Distributed Morphology (DM) → [Morphophonological Representation] →

Phonological Component (Φ) → [Phonetic Form] →

A-P

The bracketed terms name representations transferred between adjacent components.

In *generation*, the seven components are traversed in this order:

(3) **C-I** → Σ → **A-Syntax** → **A-bar Syntax** → **DM** → Φ → **A-P**

The external system C-I supplies Agentic Form to the first internal Layer, the Semantic Component Σ . From that point on, the derivation proceeds inside the Language Faculty. Each Layer receives the representation produced by the preceding component, applies its own grammar, and produces the representation required by the next component. At the end of the pipeline, the Phonological Component supplies Phonetic Form to A-P.

In *comprehension*, the same seven components are traversed in the reverse order:

(4) **A-P** → Φ → **DM** → **A-bar Syntax** → **A-Syntax** → Σ → **C-I**

In this direction, the derivation begins with Phonetic Form supplied by A-P and moves inward toward C-I. Each internal Layer reconstructs the representation required by the preceding component in the generation pipeline. Thus, comprehension does not require a second architecture. It uses the same five internal Layers, the same general processing cycle, and the same sequence of transferred representations, but traverses them in the opposite direction.

3. The Workspace as a graph

Chomsky (2019) uses the term *Workspace* for the collection of syntactic objects available during a derivation. The present proposal is related to that use of the term but gives the Workspace a specific graph-theoretic interpretation.

Merge applies to syntactic objects available in the Workspace and constructs a new syntactic object. In the present formalization, syntactic objects are represented as paths or subgraphs inside the Workspace graph. Merge is therefore interpreted as an operation that acts on paths in the graph and adds a new accessible path to the same graph.

In both familiar derivational uses of Merge and in the present proposal, Merge applies repeatedly. Each application creates a new object that can participate in later computation. The difference is that, in the present proposal, the Workspace is not represented as a single tree. It is represented as a graph that can contain several paths between the same boundary nodes.

The nodes of the graph represent minimal syntactic objects, Labels, and control points. Paths in the graph represent syntactic objects or sequences of syntactic objects that are available for further computation. Complex syntactic objects formed by Merge are represented as subgraphs of the Workspace.

When Merge applies, it extends the Workspace by adding material to the graph. In the simplest cases, it adds fresh token nodes and new edges, thereby creating an additional path between two boundary nodes. Earlier paths remain in the graph. The Workspace therefore contains the accumulated result of repeated applications of Merge.

4. Layers, Expand, and the initial Workspace

A Layer is a persistent component of the architecture, associated with its own grammars. The Workspace is the changing graph on which computation takes place.

In the present proposal, a grammar is an ordered set of Extension Rules. Each Layer is associated with two grammars: one for generation and one for comprehension.

Σ , A-Syntax, A-bar Syntax, DM, and Φ share the same general form of computation, but not the same rules.

The procedure that applies the grammar of a Layer is called Expand. Expand takes a Workspace as input and applies the ordered grammar of the Layer repeatedly, one application of Merge at a time, to that same Workspace. When no further rule application is possible, Relinearization constructs a new Workspace, which Expand returns.

The Layer is the component whose grammar is applied to that graph. The same Expand procedure is used in all five internal Layers. What differs is the grammar supplied by the Layer and by the direction of traversal.

For each Layer, the Workspace has an initial state. This is the state of the Workspace before the first application of Merge in that Layer.

The initial Workspace is the representation transferred from the previous Layer, together with the resources expected by the grammar of the present Layer, including the Label nodes used by its Extension Rules.

5. Nodes, edges, labels, and paths

The Workspace is represented here as a directed graph. The graph consists of nodes and (directed) edges.

A node is written in the form:

(6) (classname:id)

The unique key of a node is the pair consisting of its classname and its id. The id alone does not identify the node. For example:

(7) (token:1)

and

(8) (label:1)

are different nodes, because their classnames are different. Within a single graph, there is at most one node with the same pair of classname and id.

In this article, only three classnames are used: token, label, and control.

Token nodes represent positions in a path. Label nodes represent labels by which tokens are recognized by rules. In a Cognitive Graph, control nodes mark two special nodes, namely the beginning and the end of the Cognitive Graph.

The following are examples of well-formed nodes:

(9)
(token:1)
(token:2)
(label:E)
(label:F)
(control:begin)
(control:end)

In the examples in this article, token ids are written as running integers:

(10)
(token:1)
(token:2)
(token:3)

A token associated with the label E is written as a token node, such as (token:1), together with an edge from that token node to the label node (label:E).

A directed edge is indicated by an arrow:

(11) (token:1) → (token:2)

This means that there is a directed edge from (token:1) to (token:2).

A path is a sequence of nodes connected by directed edges. For example:

(12) (control:begin) → (token:1) → (token:2) → (control:end)

represents a path from (control:begin) to (control:end) through the token nodes (token:1) and (token:2).

Labelling is also represented by directed edges. For example, the following edge:

(13) (token:1) → (label:E)

means that (token:1) is associated with the label (label:E). Similarly:

(14) (token:2) → (label:F)

means that (token:2) is associated with the label (label:F).

A graph containing both the path and the labels can therefore be written explicitly as the following set of edges:

(15)

(control:begin) → (token:1)

(token:1) → (token:2)

(token:2) → (control:end)

(token:1) → (label:E)

(token:2) → (label:F)

For readability, I also use a compact curly-brace notation. An additional outgoing edge may be written immediately after the token node. This notation will usually be used for an edge from a token node to a label node.

Thus:

(16) (token:1) {label:E}

is an abbreviation for:

(17) (token:1) → (label:E)

Using this abbreviation, the graph in (15) can be written more compactly as:

(18) (control:begin) → (token:1) {label:E} → (token:2) {label:F} → (control:end)

The curly-brace notation is a compact way of displaying an additional outgoing edge from the source token node to the label node named inside the braces.

6. Extension Rules and a simple Merge example

Rules retain familiar linguistic notation:

(19) Left => Right

The interpretation is:

(20) find Right, add Left

Thus, a rule such as:

(21) $A \Rightarrow B C$

is applied by finding a path labelled $B C$ in the Workspace and adding a path labelled A at the same location.

The rule does not replace $B C$ with A . Instead, it extends the Workspace so that A becomes available as an additional path. In this sense, Merge is interpreted as graph extension.

Assume the initial Workspace contains the following graph:

(22) $(\text{control:begin}) \rightarrow (\text{token:1}) \{\text{label:B}\} \rightarrow (\text{token:2}) \{\text{label:C}\} \rightarrow (\text{control:end})$

This graph contains a path from (control:begin) to (control:end) through two token nodes. The first token node is labelled B , and the second token node is labelled C . Thus, the path from (control:begin) to (control:end) contains the label sequence $B C$.

Now suppose the Layer has the following Extension Rule:

(23) $A \Rightarrow B C$

The right side of the rule is found in the Workspace:

(24) $B C$

The matching span is:

(25) $(\text{token:1}) \{\text{label:B}\} \rightarrow (\text{token:2}) \{\text{label:C}\}$

Merge then adds a fresh token node labelled A , creating an additional path between the same boundary nodes:

(26) $(\text{control:begin}) \rightarrow (\text{token:3}) \{\text{label:A}\} \rightarrow (\text{control:end})$

After Merge, the Workspace contains both paths:

(27)

$(\text{control:begin}) \rightarrow (\text{token:1}) \{\text{label:B}\} \rightarrow (\text{token:2}) \{\text{label:C}\} \rightarrow (\text{control:end})$

$(\text{control:begin}) \rightarrow (\text{token:3}) \{\text{label:A}\} \rightarrow (\text{control:end})$

The two paths are alternative paths between the same boundary nodes. The original path has not been erased. It remains in the Workspace, while the newly added path makes A available for later applications of Merge.

In this example, the boundary nodes are (control:begin) and (control:end). More generally, Merge can apply between any two nodes X and Y, provided there is a path from X to Y matching the right side of an Extension Rule. Merge finds such a path and adds another path between those same boundary nodes.

The same interpretation applies in both directions of traversal. There are different ordered sets of Extension Rules for generation and comprehension. The direction affects Relinearization and Transfer, but not the basic form of Merge as graph extension.

7. Merge inside a larger Workspace

The previous example used (control:begin) and (control:end) as the boundary nodes. That is only the simplest case. In general, Merge can apply inside a larger Workspace.

Consider a simplified but more grammar-like example. Suppose the Workspace contains the following path:

(28)

(control:begin) → (token:1) {label:pronoun} → (token:2) {label:verb} → (token:3) {label:Det} → (token:4) {label:noun} → (control:end)

Suppose also that the Layer contains the Extension Rule:

(29) DP => Det noun

For this application of Merge, the relevant boundary nodes are:

(30) X = (token:2)

Y = (control:end)

Between these two boundary nodes, the Workspace contains a path whose relevant label sequence is:

(31) Det noun

The matching span is:

(32) (token:3) {label:Det} → (token:4) {label:noun}

Merge then adds a fresh token node labelled DP and creates an additional path between the same boundary nodes:

(33) (token:2) → (token:5) {label:DP} → (control:end)

After this application of Merge, the Workspace contains both the original path:

(34) (token:2) {label:verb} → (token:3) {label:Det} → (token:4) {label:noun} → (control:end)

and the newly added path:

(35) (token:2) {label:verb} → (token:55) {label:DP} → (control:end)

The Workspace is not limited to the material directly involved in the current application of Merge. It contains all accumulated objects in the graph.

A particular application of Merge is localized by two boundary nodes, searches for the right side of a rule between those boundaries and adds the left side as a new path between the same boundaries.

8. Layer grammars and Expand

Each Layer has its own grammars. In the present proposal, the grammar of a Layer is an ordered set of Extension Rules. The order matters, because Merge operations are applied by scanning the rules of the relevant grammar.

Each Layer has two grammars: one for generation and one for comprehension. The generation grammar is used when the architecture is traversed from C-I toward A-P. The comprehension grammar is used when the architecture is traversed from A-P toward C-I. The comprehension grammar is therefore parsing-like, but it belongs to the same Layer and uses the same general form of Extension Rules.

What changes from Layer to Layer is the grammar: the ordered set of Extension Rules available in that Layer for the relevant direction of traversal. What remains the same is the processing procedure. This procedure is called Expand.

Internally, Expand proceeds as follows:

- Receive a Workspace.
- Select the grammar of the present Layer for the current direction of traversal.

- Use the selected ordered grammar to apply Merge repeatedly inside the Workspace.
- Relinearize the resulting Workspace.
- Return a new graph as the Output Graph.

The Workspace received by a Layer contains both (a) the material transferred from the previous Layer, and (b) the Label nodes expected by the grammar of the current Layer. All the Label nodes are already present when the first Merge applies.

A particular application of Merge may introduce a fresh token node and link it to an existing Label node, but it does not need to create the Label node at that moment.

Thus, a Layer is defined by its position in the architecture and by the two grammars associated with it: one for generation and one for comprehension. For a given traversal, Expand selects the appropriate grammar and applies it to the current Workspace. The processing procedure itself remains the same across all five internal Layers.

9. Merge can be understood as graph extension

The proposal is that Merge itself can be formally interpreted as graph extension inside the Workspace.

In the Workspace conception of Merge, objects already available in the Workspace are used to make a new object available. In the graph interpretation, this means that a path already available in the Workspace is used to add another path between the same boundary nodes.

Consider a simple lexical path:

(36) (control:begin) → (token:100) {label:the} → (token:101) {label:book} → (control:end)

Here the words the and book are not token ids. They are labels. Written without the curly-brace abbreviation, the relevant label edges are:

(37)

(token:100) → (label:the)

(token:101) → (label:book)

Now suppose the Layer contains the following Extension Rules:

(38)

Det => the

noun => book

DP => Det noun

The first rule finds a token labelled the and adds a new token labelled Det over the same span. For example, it may add:

(39) (control:begin) → (token:102) {label:Det} → (token:101)

The original token (token:100) is not removed. The Workspace now contains both the original lexical path through (token:100) and the new path through (token:102).

The second rule finds a token labelled book and adds a new token labelled noun over the same span. For example, after the first application, it may add:

(40) (token:102) → (token:103) {label:noun} → (control:end)

At this point, the Workspace contains a path whose relevant label sequence is:

(41) Det noun

That path is:

(42) (control:begin) → (token:102) {label:Det} → (token:103) {label:noun} → (control:end)

The third rule can now apply:

(43) DP => Det noun

Merge finds the path labelled Det noun and adds a fresh token node labelled DP between the same boundary nodes:

(44) (control:begin) → (token:104) {label:DP} → (control:end)

After these applications, the Workspace contains several paths, including the original lexical path:

(45) (control:begin) → (token:100) {label:the} → (token:101) {label:book} → (control:end)

the category path:

(46) (control:begin) → (token:102) {label:Det} → (token:103) {label:noun} → (control:end)

and the newly added DP path:

(47) (control:begin) → (token:104) {label:DP} → (control:end)

The important point is that Merge does not replace one path with another. The lexical path remains in the Workspace. The category path is added. The DP path is then added as another accessible path between the same boundary nodes.

After a new path is added, that path may make further Extension Rules applicable. Expand therefore continues as long as further applications of the Layer's Extension Rules remain possible.

This gives a formal interpretation of the basic intuition behind Merge: objects available in the Workspace are used to create another object, also available in the same Workspace.

10. Relinearization and Transfer

Relinearization is the construction of a new Output Graph from selected paths in the extended Workspace.

After Merge operations have extended the Workspace, several paths may be available between the same boundary nodes. Some of these paths may be interpretable by the next Layer; others may not. Relinearization determines which paths and labels are passed on.

Relinearization walks complete paths from (control:begin) to (control:end), extracts the output labels that are interpretable by the next Layer, and builds a new graph from them.

The Output Graph is a new Workspace. It is not the same Workspace with some paths marked as visible or invisible. It may contain equivalently named nodes, including equivalently named label nodes and control nodes, but it is a new graph object constructed by Relinearization.

Transfer is the passing of the Output Graph of one Layer as the initial Workspace of the next Layer.

When the next Layer receives this Workspace, the transferred material supplies the initial state of that Layer. All the Label nodes expected by the grammar of the next Layer are also present in that Workspace before Merge begins.

Thus, Expand extends the Workspace by repeated applications of Merge, Relinearization constructs a new Output Graph from the extended Workspace, and Transfer passes that Output Graph as the initial Workspace for the next Layer.

11. Conclusion

The five internal Layers can be traversed in two directions.

In *generation*, the order is:

(48) $\Sigma \rightarrow \mathbf{A-Syntax} \rightarrow \mathbf{A-bar Syntax} \rightarrow \mathbf{DM} \rightarrow \Phi$

Generation begins with a Workspace supplied by C-I to the Semantic Component and proceeds toward A-P.

In *comprehension*, the order is reversed:

(49) $\Phi \rightarrow \mathbf{DM} \rightarrow \mathbf{A-bar Syntax} \rightarrow \mathbf{A-Syntax} \rightarrow \Sigma$

Comprehension begins with a Workspace supplied from the A-P side and proceeds toward C-I.

The grammars differ from Layer to Layer, and they also differ by direction. The ordered set of Extension Rules used by a Layer in generation is not expected to be identical to the ordered set of Extension Rules used by that Layer in comprehension.

Nevertheless, the algorithm remains the same: Expand, Relinearization, Transfer.

In both directions, a Layer receives a Workspace, applies the relevant ordered grammar, constructs a new Output Graph, and transfers that graph to the next Layer.

References

Chomsky, Noam. 2019. *The UCLA lectures* (April 29–May 2, 2019). Transcribed and edited by Robert Freidin. LingBuzz/005485.

Halle, Morris, and Alec Marantz. 1993. Distributed Morphology and the pieces of inflection. In *The view from Building 20: Essays in honor of Sylvain Bromberger*, ed. Kenneth Hale and Samuel Jay Keyser, 111–176. Cambridge, MA: MIT Press.

Margaliot, Sasson. 2017. *Minimalist semantics and pre-linguistic intelligence*. LingBuzz/003409.